UNIVERSIDAD POLITÉCNICA DE PACHUCA

Ing. en Tecnologías De La Información Y La Innovación Digital

REPORTE DE PRÁCTICA

Estructuración de interfaces con Scaffold: Implementación de Toolbar y FAB en Jetpack Compose

Asignatura: Desarrollo de aplicaciones móviles

Alumno: Lorenzana Hernández Aarón Dahir

Matrícula: 2531126341

Grupo: TIID 4-02

Profesor(a): MIGUEL ANGEL MONTOYA CERRO

Pachuca de Soto, Hidalgo, México, 13/10/2026

Índice de contenido

1. Introducción 1

1.1 Contextualización 1

1.2 Planteamiento del problema u objeto de estudio 1

1.3 Objetivos 2

1.4 Justificación 2

1.5 Estructura del documento 3

2. Marco teórico 4

2.1 Definición de conceptos clave 4

2.2 Teorías y modelos relevantes 5

2.3 Antecedentes 6

2.4 Fuentes de información 6

3. Desarrollo 7

3.1 Análisis de la vista sin contenedor Scaffold 7

3.2 Implementación del componente Scaffold 9

3.3 Creación e integración de la Toolbar (TopAppBar) 11

3.4 Creación e integración del Floating Action Button (FAB) 13

3.5 Manejo de eventos con LocalContext y Toast 15

3.6 Análisis de resultados 17

4. Conclusiones 18

4.1 Aportes 18

4.2 Limitaciones 18

4.3 Recomendaciones y proyecciones 18

5. Referencias 19

6. Anexos 20

Anexo A. Datos del video consultado 20

Índice de ilustraciones

Ilustración 1: Árbol del proyecto en Android Studio con MainActivity.kt 7

Ilustración 2: Código inicial de la vista App() utilizando un LazyColumn suelto 8

Ilustración 3: Vista en el emulador sin barra superior ni estructura definida 8

Ilustración 4: Integración del componente Scaffold como esqueleto principal 9

Ilustración 5: Creación de la función Composable Toolbar() con TopAppBar 10

Ilustración 6: Emulador mostrando la interfaz actualizada con la Toolbar gris 10

Ilustración 7: Importación del LocalContext para el uso de notificaciones 11

Ilustración 8: Creación de la función FAB() con FloatingActionButton 11

Ilustración 9: Implementación de Toast.makeText dentro del evento onClick 12

Ilustración 10: Emulador mostrando el botón flotante en la esquina inferior derecha 12

Ilustración 11: Asignación de Toolbar y FAB en las propiedades del Scaffold 13

Ilustración 12: Implementación de paddingValues para evitar solapamientos 14

Ilustración 13: Emulador mostrando el Toast "Suscríbete" al presionar el FAB 15

Ilustración 14: Código final integrando los componentes personalizados en App() 16

Ilustración 15: Configuración final en onCreate llamando al contenedor estructurado 16

Índice de tablas

Tabla 1: Comparación de contenedores entre sistema XML y Jetpack Compose 5

Tabla 2: Componentes principales del Scaffold 7

Tabla 3: Beneficios del uso del Scaffold en el desarrollo de interfaces 15

1. Introducción

1.1 Contextualización

Al desarrollar aplicaciones nativas para Android mediante el framework declarativo Jetpack Compose, las vistas se construyen componiendo múltiples funciones. Sin embargo, apilar simples `Column` o `LazyColumn` resulta en una interfaz carente de los elementos estándar de navegación que los usuarios de Android esperan. Para solventar esta necesidad de estructura, Material Design provee patrones arquitectónicos visuales.

El componente encargado de proveer este molde o esqueleto visual es el `Scaffold`. Este contenedor abstrae la complejidad de alinear correctamente barras de navegación, botones flotantes y menús laterales, permitiendo al desarrollador enfocarse en el contenido interno de la aplicación.

1.2 Planteamiento del problema u objeto de estudio

El problema a abordar en esta práctica es la desestructuración de una interfaz gráfica que fue programada únicamente con layouts básicos. Según el tutorial de AristiDevs (Guimerá Orozco, 2021), una aplicación sin una barra superior (Toolbar) parece simplemente una "vista sacada de contexto" en lugar de una aplicación completa.

El objeto de estudio es la implementación del componente `Scaffold` para estructurar la aplicación, agregando una barra superior (`TopAppBar`) y un botón de acción flotante (`FloatingActionButton`), además de garantizar que estos elementos no se sobrepongan al contenido principal (`LazyColumn`).

1.3 Objetivos

Objetivo general. Reestructurar una vista básica en Jetpack Compose utilizando el componente Scaffold para integrar elementos estándar de Material Design y mejorar la experiencia de usuario.

Objetivos específicos.

1. Analizar una vista básica conformada únicamente por un `LazyColumn`.
2. Implementar el componente `Scaffold` como contenedor principal de la aplicación.
3. Crear y personalizar una barra superior mediante el uso de `TopAppBar`.
4. Integrar un botón de acción flotante (`FloatingActionButton`) configurado para disparar un evento.
5. Utilizar `LocalContext.current` para desplegar mensajes emergentes (`Toast`) dentro de Compose.
6. Aplicar correctamente los `paddingValues` provistos por el Scaffold para evitar oclusiones en la UI.

1.4 Justificación

Académicamente, dominar los contenedores de Material Design como `Scaffold` es fundamental para el diseño de interfaces estandarizadas. Permite a los estudiantes construir aplicaciones que respeten las guías de interfaz humana (HIG) de Google, un requisito indispensable en el desarrollo móvil contemporáneo.

A nivel profesional, el uso de estos moldes prefabricados acelera significativamente los tiempos de desarrollo y reduce la cantidad de código repetitivo. Evita dolores de cabeza relacionados con la superposición de elementos, ya que el sistema calcula automáticamente los márgenes necesarios para que el contenido no quede oculto detrás de las barras del sistema o la botonera.

1.5 Estructura del documento

Este reporte se divide en secciones progresivas. Tras la introducción, el marco teórico explica el paradigma de los contenedores en Compose frente a los antiguos XML. El desarrollo detalla la integración paso a paso del Scaffold, Toolbar y FAB con sus respectivas capturas. Finalmente, se presentan las conclusiones, aprendizajes y referencias consultadas.

2. Marco teórico

En esta sección se definen los componentes estructurales de Material Design que facilitan el desarrollo de interfaces nativas modernas.

2.1 Definición de conceptos clave

Scaffold. En Jetpack Compose, Scaffold es una función composable que implementa la estructura visual de diseño básica de Material Design. Proporciona "ranuras" (slots) específicas para los componentes más comunes de nivel superior en la pantalla, asegurando que se posicionen y comporten adecuadamente unos respecto a otros.

TopAppBar (Toolbar). Es un contenedor dedicado a la parte superior de la pantalla. Generalmente muestra información y acciones relacionadas con la vista actual, como el título de la página, un botón de retroceso o iconos de acciones (buscar, menú).

Floating Action Button (FAB). Es un botón circular que flota sobre el contenido para promover una acción primaria en la aplicación (como agregar un nuevo elemento, escribir un correo o iniciar una descarga).

PaddingValues. Es un objeto inyectado por el Scaffold que contiene los valores numéricos de los márgenes que el contenido principal debe respetar para no quedar cubierto por la TopAppBar superior o la barra de navegación inferior.

2.2 Teorías y modelos relevantes

Slot API (API de Ranuras). Compose utiliza un patrón de diseño llamado *Slot API*. En lugar de obligar al desarrollador a pasar una multitud de parámetros para configurar un componente, el componente expone ranuras vacías (parámetros que reciben funciones `@Composable`) donde el desarrollador puede inyectar el código que desee. Scaffold es el mayor exponente de este patrón, como se compara en la Tabla 1.

Tabla 1: Comparación de contenedores entre sistema XML y Jetpack Compose

Característica	Sistema XML (Imperativo)	Jetpack Compose (Declarativo)
Contenedor base	`CoordinatorLayout`	`Scaffold`
Barra superior	`AppBarLayout` + `Toolbar`	Ranura `topBar = { TopAppBar(...) }`
Botón principal	Declarado en XML con atributos `anchor`	Ranura `floatingActionButton = { ... }`
Manejo de superposición	Atributo `app:layout_behavior`	Aplicación manual de `paddingValues`

2.3 Antecedentes

En las versiones anteriores de Android basadas en Java y XML, lograr que un botón flotara y se ocultara automáticamente al hacer scroll, o que el contenido no se solapara con la Action Bar, requería una amalgama compleja de CoordinatorLayouts, AppBarLayouts, anclajes y comportamientos definidos por XML. Compose abstrae toda esa física en parámetros sencillos pasados a una única función.

3. Desarrollo

A continuación se detallan los pasos realizados para refactorizar la vista principal, pasando de un conjunto de componentes apilados a una arquitectura visual limpia de Material Design.

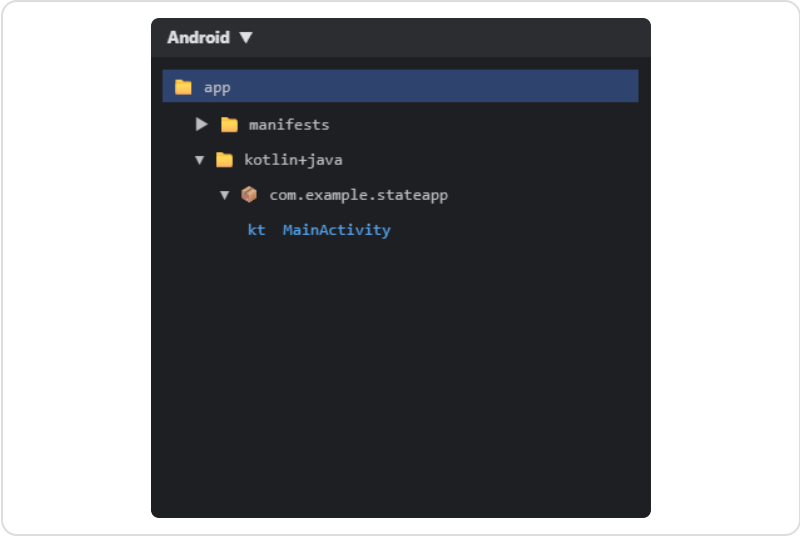


Ilustración 1: Árbol del proyecto en Android Studio con MainActivity.kt

3.1 Análisis de la vista sin contenedor Scaffold

El estado inicial del proyecto consistía en una función `App()` que retornaba directamente un `LazyColumn`. Aunque funcional, esta distribución carecía de contexto de aplicación, presentando el contenido adherido al borde superior sin identidad (Ilustración 2 y 3).



Ilustración 2: Código inicial App()



Ilustración 3: Vista sin contenedor

3.2 Implementación del componente Scaffold

Tal como indica Guimerá Orozco (2021), el primer paso es envolver nuestro contenido con el componente `Scaffold`. Para mantener el código atómico y legible, en lugar de declarar la barra superior y el botón dentro de los parámetros, se procedió a delegar esas responsabilidades a nuevas funciones externas (Ilustración 4).



Ilustración 4: Integración del componente Scaffold como esqueleto principal

3.3 Creación e integración de la Toolbar (TopAppBar)

Se definió la función `@Composable fun Toolbar()` implementando el componente `TopAppBar`. Se configuró el texto del título y se le aplicó un color oscuro para que concordara con la temática negra que tenía la aplicación originalmente (Ilustración 5 y 6).

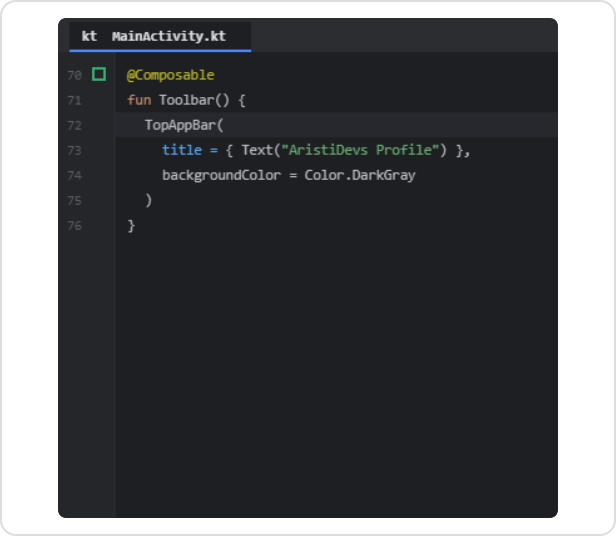


Ilustración 5: Creación Toolbar()



Ilustración 6: Toolbar en emulador

3.4 Creación e integración del Floating Action Button (FAB)

El siguiente requerimiento era agregar un botón flotante. Se creó la función `@Composable fun FAB()`. Sin embargo, para que el botón disparara un mensaje en pantalla (Toast), era indispensable obtener el contexto de la aplicación, el cual ya no se accede con el clásico `this` o `getContext()`, sino a través de la provisión estática `LocalContext.current` (Ilustración 7 y 8).



Ilust. 7: Uso LocalContext



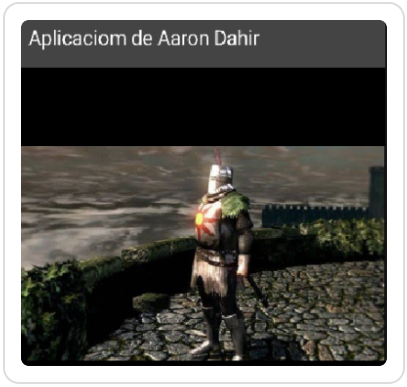
Ilust. 8: Función FAB()

3.5 Manejo de eventos con LocalContext y Toast

Una vez obtenido el contexto de la interfaz actual, se programó el evento `onClick` del botón para invocar a `Toast.makeText`. Se asignó el botón al Scaffold, que automáticamente lo posicionó en la esquina inferior derecha gracias a sus parámetros por defecto (Ilustraciones 9, 10 y 11).



Ilust. 9: Evento Toast



Ilust. 10: FAB posicionado



Ilust. 11: Asignación en Scaffold

Finalmente, para asegurar que la lista principal (`LazyColumn`) no iniciara escondida detrás de la Toolbar, se aplicó la variable `paddingValues` que provee el bloque de contenido del Scaffold directamente al modificador de la columna. Al probar la aplicación, la interfaz reaccionó perfectamente al toque del botón, mostrando el Toast "Suscríbete" (Ilustración 12 y 13).

```kt MainActivity.kt
    ) { paddingValues ->
      LazyColumn(
        modifier = Modifier
          .fillMaxSize()
          .padding(paddingValues)
      ) {
```

Ilustración 12: Uso de paddingValues

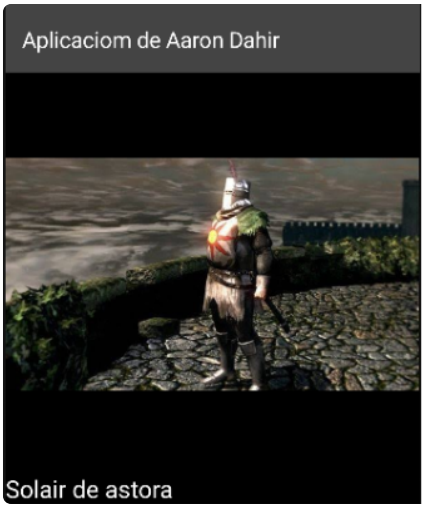


Ilustración 13: Toast funcionando

3.6 Análisis de resultados

El resultado es una aplicación que respeta los estándares de usabilidad de Android, implementada con muy pocas líneas de código gracias al poder del *Slot API* (Ilustraciones 14 y 15). Se demostró que Jetpack Compose simplifica el desarrollo al eliminar la necesidad de configurar anclajes manuales, manejando el *layout* y las superposiciones a través del delegado Scaffold.

```kt MainActivity.kt
        Greeting(name = "Aaron")

        MyRowExample()
      }
    }
  }
```

Ilustración 14: Código final App()

```kt MainActivity.kt
class MainActivity : ComponentActivity() {
  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent {
      App()
    }
  }
}
```

Ilustración 15: Función MainActivity

4. Conclusiones

4.1 Aportes

Esta práctica aportó un entendimiento profundo sobre cómo organizar pantallas en Compose. Se aprendió que el `Scaffold` no es un mero contenedor, sino un administrador inteligente de espacios que distribuye y aplica márgenes (`paddingValues`) para que barras de navegación y contenido armonicen sin solaparse. Además, se asimiló la técnica de obtener el contexto del sistema mediante `LocalContext.current`.

4.2 Limitaciones

La práctica se limitó al uso del Scaffold básico de Material 2/3 con barra superior y botón flotante. No se implementaron otros componentes del Scaffold como el menú de navegación inferior (`BottomNavigation`) o el menú lateral (`NavigationDrawer`), los cuales añaden mayor complejidad a la gestión del estado.

4.3 Recomendaciones y proyecciones

Como siguiente paso, se recomienda explorar el uso del `ScaffoldState` (o `DrawerState` en Material 3) para controlar programáticamente la apertura y cierre del menú de navegación lateral. También es aconsejable abstraer la navegación a un `NavHost` para que la barra superior actualice dinámicamente su título dependiendo de la pantalla activa.

5. Referencias

Android Developers. (s. f.). *Diseño de componentes de Material en Compose*. Google. https://developer.android.com/develop/ui/compose/layouts/material?hl=es-419

Guimerá Orozco, A. [Programación by AristiDevs]. (2021, 14 de noviembre). *Scaffold (TOOLBAR y FAB) en JETPACK COMPOSE - Tutorial Android JETPACK COMPOSE desde cero [2021]* [Video]. YouTube. https://www.youtube.com/watch?v=jvfZWtoggwc

6. Anexos

Anexo A. Datos del video consultado

Título: Scaffold (TOOLBAR y FAB) en JETPACK COMPOSE - Tutorial Android JETPACK COMPOSE desde cero [2021]

Canal: Programación by AristiDevs (Mobile Dev by AristiDevs)

Enlace al video: https://www.youtube.com/watch?v=jvfZWtoggwc

Fecha de publicación original: 14 de noviembre de 2021

Temas técnicos abordados: Implementación del molde `Scaffold`, creación de `TopAppBar` (Toolbar), implementación de `FloatingActionButton` (FAB), uso de `LocalContext.current` y manejo de `Toast`.